

Memoria Técnica — MusicPlayer

Resumen

MusicPlayer es una plataforma web de reproducción y gestión de música desarrollada como Trabajo de Fin de Grado. Mi parte del proyecto se centra en el backend: el diseño de la API REST, la implementación de la seguridad mediante tokens JWT, el modelo de datos relacional, el sistema de migraciones con Liquibase y el servicio de subida y streaming de archivos de audio e imagen.

El sistema permite a los usuarios escuchar música en tiempo real, gestionar listas de reproducción, seguir artistas, marcar canciones como favoritas, buscar contenido y consultar letras de canciones en tiempo real. La arquitectura está claramente separada: el frontend Angular se comunica con el backend Spring Boot exclusivamente a través de HTTP, y el backend gestiona toda la lógica de negocio, el acceso a la base de datos MySQL y el streaming de archivos de audio.

Este documento describe el proceso completo de desarrollo del backend: las decisiones de diseño tomadas, los problemas concretos que aparecieron durante la implementación y cómo se resolvieron. También incluye la planificación temporal y financiera del proyecto.

Resumen extendido

MusicPlayer es una plataforma web de streaming y gestión musical desarrollada como Trabajo de Fin de Grado. Mi contribución se centra en la capa de backend: diseño de la API REST, seguridad mediante JWT, el modelo de datos relacional y el sistema de migraciones con Liquibase.

El sistema permite a los usuarios reproducir música en tiempo real, gestionar listas de reproducción, seguir artistas y consultar letras de canciones en directo. La arquitectura está claramente separada: el frontend en Angular se comunica con el backend Spring Boot exclusivamente a través de HTTP, mientras que el backend gestiona toda la lógica de negocio y el acceso a la base de datos MySQL.

Este documento describe el proceso completo de desarrollo del backend, desde las decisiones de diseño iniciales hasta los problemas concretos que aparecieron durante la implementación y cómo se resolvieron. También recoge la planificación temporal y financiera del proyecto.

Índice

1. BLOQUE 1: Descripción del Proyecto

- B1.1 Introducción y Motivación
- B1.2 Objetivos del Proyecto
- B1.3 Estado del Arte
- B1.4 Elicitación de Requisitos y Análisis de Riesgos

2. BLOQUE 2: Ejecución del Proyecto

- B2.1 Diseño del Sistema
- B2.2 Implementación
 - B2.2.1 Tecnologías Empleadas
 - B2.2.2 Desarrollo
- B2.3 Pruebas del Sistema

3. BLOQUE 3: Planificación del Proyecto

- B3.1 Planificación Temporal Inicial
- B3.2 Planificación Financiera Inicial
- B3.3 Planificación Temporal Final (Real)
- B3.4 Planificación Financiera Final (Real Parcial)
- B3.5 Estudio de Mercado

4. Conclusiones

5. Trabajo Futuro

6. Bibliografía

7. Anexos

BLOQUE 1: DESCRIPCIÓN DEL PROYECTO

B1.1 Introducción y Motivación

El contexto: cómo escuchamos música hoy

En poco más de dos décadas, la forma en que las personas consumen música ha cambiado de manera radical, casi sin que nos hayamos dado cuenta. A principios de los años 2000, escuchar música implicaba ir a una tienda, comprar un CD, o en el peor de los casos, descargar archivos MP3 de plataformas de dudosa legalidad. Hoy, más de 700 millones de personas pagan una suscripción mensual para acceder a catálogos de millones de canciones desde su móvil, su ordenador o el altavoz de la cocina.

Este cambio no es solo tecnológico. Es cultural. La música ha dejado de ser algo que se posee —un disco físico en una estantería— para convertirse en un servicio al que se accede. Nadie colecciona álbumes en MP3 ya; la gente sigue artistas en Spotify, crea playlists de boda en Apple Music o descubre grupos nuevos a través de los algoritmos de YouTube Music.

Detrás de toda esa experiencia, aparentemente sencilla para el usuario final, hay una infraestructura técnica enorme: bases de datos con millones de entradas, sistemas de autenticación y autorización, APIs públicas y privadas, reproductores web y móviles, y mecanismos de streaming adaptativo que ajustan la calidad del audio en tiempo real según el ancho de banda disponible.

Cuando uno se pone a pensar en todo lo que hay que construir para que algo así funcione, la cabeza empieza a girar. Pero también aparece una pregunta interesante: ¿se puede construir algo parecido —aunque sea a escala mucho más pequeña— con las herramientas que se aprenden durante un grado de ingeniería informática? La respuesta, que es la que motivó este proyecto, es que sí se puede.

Por qué elegimos una plataforma de streaming musical

Cuando mi compañero y yo nos pusimos a pensar en qué queríamos hacer para el TFG, teníamos claro un par de cosas desde el principio. Primero, queríamos construir algo real, no un ejercicio académico de laboratorio que nadie usaría jamás. Segundo, queríamos que el proyecto nos obligase a integrar la mayor parte de lo que habíamos aprendido durante el grado: diseño de bases de datos, arquitecturas REST, seguridad de aplicaciones, desarrollo frontend reactivo y consumo de APIs externas.

Una plataforma de streaming musical cumple todos esos requisitos mejor que casi cualquier otro tipo de proyecto. Necesita un modelo de datos relacional no trivial con entidades interconectadas: canciones, álbumes, artistas,

géneros, playlists, usuarios, reproducciones. Requiere un sistema de autenticación robusto con distintos niveles de acceso. Necesita una interfaz de usuario atractiva que justifique el uso de un framework frontend moderno. Y permite integrar servicios externos para añadir valor sin tener que construir todo desde cero.

Además, y esto es algo que suele pasarse por alto en los proyectos académicos: una plataforma de música es algo que cualquier persona puede entender y evaluar sin necesidad de conocimientos técnicos. Cuando alguien mira MusicPlayer, ve una aplicación que reconoce, con una barra lateral con la biblioteca, una barra de reproducción en la parte de abajo, una lista de canciones. No necesita que le expliquen para qué sirve.

La problemática que resuelve

Spotify, Apple Music, YouTube Music, Deezer y Tidal dominan el mercado del streaming musical. Todas ellas comparten una característica fundamental: son sistemas cerrados, de código propietario, diseñados para funcionar a escala de decenas de millones de usuarios, y con modelos de negocio basados en licencias de contenido y suscripciones.

Ninguna de ellas está pensada para ser desplegada de forma autogestionada por una organización pequeña. Una escuela de música que quiera poner a disposición de sus alumnos las grabaciones de los conciertos del año no tiene ninguna opción en ese mercado. Un sello discográfico independiente que quiera ofrecer su catálogo directamente a sus fans sin intermediarios tampoco. Un colectivo cultural que organice festivales de música local y quiera archivar y distribuir sus grabaciones, tampoco.

MusicPlayer cubre ese espacio. Es una plataforma de streaming musical pensada para ser desplegada y gestionada por organizaciones que quieren controlar su propio catálogo y sus propios usuarios, sin depender de terceros ni pagar licencias de uso. La arquitectura de roles permite que la organización gestione el sistema (administrador), que un equipo editorial gestione el catálogo (editor), que los propios artistas suban y gestionen su contenido (artista) y que los oyentes disfruten del resultado (usuario).

Cómo nos dividimos el trabajo

Como el proyecto lo desarrollamos entre dos personas, desde el principio acordamos una división clara de responsabilidades para que cada memoria pudiera ser genuinamente independiente. Mi responsabilidad fue el backend: el diseño del modelo de datos, la implementación de la API REST con Spring Boot, la seguridad con JWT, el sistema de migraciones Liquibase y la configuración del servidor. Mi compañero se encargó del frontend: los componentes Angular, el diseño visual, el tema Spotify-style y la integración con el backend desde el lado del cliente.

Esta división tiene sentido también desde el punto de vista del proyecto en sí: el backend y el frontend son capas completamente desacopladas que se comunican a través de un contrato HTTP bien definido. Lo que ocurre dentro de cada capa es, en su mayor parte, independiente de la otra. Eso facilitó que pudiéramos trabajar en paralelo sin pisarnos constantemente.

Alcance del proyecto

La versión actual implementa el ciclo completo de una plataforma musical operativa: registro y activación de cuentas por correo electrónico, autenticación con JWT, dashboards diferenciados por rol, gestión del catálogo musical (canciones, álbumes, artistas, géneros), listas de reproducción personalizadas, historial de reproducciones, canciones favoritas, buscador de canciones y consulta de letras en tiempo real. El reproductor de audio está completamente implementado con reproducción real, cola de canciones, modos aleatorio y repetición, control de progreso y volumen. El backend dispone de un servicio de subida y streaming de archivos de audio e imagen que almacena los archivos localmente y los sirve directamente al reproductor del frontend.

B1.2 Objetivos del Proyecto

Objetivos técnicos

El primer objetivo fue diseñar e implementar una API REST completa que cubriese todas las operaciones del dominio musical. No me conformé con hacer únicamente los endpoints más simples: quería que la API fuese lo suficientemente completa como para que el frontend pudiese construir sobre ella una experiencia de usuario real, con paginación, filtrado, creación, edición y eliminación de recursos.

El segundo objetivo fue implementar un sistema de autenticación y autorización robusto. JWT era la elección natural para una arquitectura desacoplada como la nuestra: el servidor no necesita mantener sesiones en memoria, y el cliente puede incluir el token en cada petición sin depender de cookies. La parte de autorización, con cuatro roles distintos y control granular de permisos por endpoint, fue uno de los aspectos más complejos de implementar, como explico más adelante.

El tercer objetivo fue garantizar la integridad y reproducibilidad del esquema de base de datos mediante Liquibase. Que dos entornos distintos (mi máquina de desarrollo, el servidor de producción, la máquina de mi compañero) pudiesen arrancar la aplicación y tener exactamente la misma base de datos, sin intervención manual, era un requisito que me parecía innegociable desde el principio.

El cuarto objetivo fue integrar el sistema de correo electrónico para la activación de cuentas y la recuperación de contraseñas. Este componente se obvia habitualmente en los proyectos académicos porque complica la configuración del entorno, pero en una aplicación real es absolutamente necesario.

El quinto objetivo fue documentar el backend de forma que cualquier desarrollador pudiese entender la arquitectura, configurar el entorno y extender la funcionalidad sin necesidad de consultar al equipo original.

Objetivos formativos

Trabajar con JHipster fue revelador. Antes de este proyecto, pensaba que configurar Spring Boot con Spring Security era cuestión de seguir los tutoriales de la documentación oficial. Lo que JHipster me mostró es que una configuración real, lista para producción, es mucho más compleja: filtros de seguridad, interceptores CORS, generación y validación de tokens, gestión de errores coherente en todas las capas. Ver todo eso generado y funcionando me dio un punto de referencia muy valioso.

También me propuse aprender a trabajar con Spring Data JPA en situaciones que van más allá de los casos de uso básicos: relaciones ManyToMany con consultas JPQL, auditoría automática y el patrón repositorio con relaciones bag. Estas son cosas que raramente se cubren en los cursos de Spring, pero que aparecen en cualquier proyecto real con un modelo de datos no trivial.

B1.3 Estado del Arte

El mercado del streaming musical está dominado por unas pocas plataformas comerciales. Antes de definir el diseño de MusicPlayer, estudié las principales alternativas disponibles para entender qué ofrecen, qué limitaciones tienen y en qué punto podría diferenciarse nuestro proyecto.

Spotify

Spotify es la plataforma de streaming musical más utilizada del mundo, con más de 600 millones de usuarios activos mensuales a principios de 2024. Lanzada en 2008 por dos emprendedores suecos, revolucionó la industria con un modelo freemium que permite acceder a todo el catálogo de forma gratuita con publicidad, o sin publicidad y con funciones adicionales mediante suscripción.

Técnicamente, Spotify es un sistema extraordinariamente complejo. Su infraestructura de backend utiliza microservicios desplegados en Google Cloud, con sistemas de caché distribuida para minimizar la latencia, y

algoritmos de recomendación basados en aprendizaje automático que analizan el historial de escucha de cada usuario. La experiencia de usuario que resulta de todo eso es muy pulida: interfaz oscura y limpia, playlists generadas automáticamente como "Discover Weekly", podcasts integrados y funcionalidades sociales.

Para el caso de uso de MusicPlayer, Spotify tiene una limitación fundamental: es un sistema completamente cerrado. No existe versión autogestionable, los artistas solo pueden subir contenido a través de distribuidores específicos, y las organizaciones no pueden desplegar su propia instancia. La API de Spotify para desarrolladores existe, pero ha experimentado restricciones importantes en los últimos años que han afectado a muchos proyectos de terceros.

Lo que MusicPlayer aporta frente a Spotify: autogestionabilidad total. Una organización puede desplegar MusicPlayer en su propio servidor, gestionar sus propios usuarios y controlar exactamente qué contenido está disponible, sin depender de ningún tercero.

YouTube Music

YouTube Music es la apuesta de Google por el streaming musical, lanzada en su forma actual en 2018 tras la desaparición de Google Play Music. Su mayor ventaja competitiva es el catálogo: al integrar el contenido de YouTube, incluye versiones en directo, maquetas, grabaciones de baja calidad subidas por fans y contenido raro que no está disponible en plataformas con catálogos estrictamente licenciados.

La integración con el ecosistema Google es potente, pero la experiencia de usuario es notablemente menos cuidada que la de Spotify. La mezcla entre contenido de alta calidad y vídeos de YouTube de cualquier origen puede resultar inconsistente y confusa. Al igual que Spotify, es un sistema propietario cerrado.

Lo que MusicPlayer aporta frente a YouTube Music: control de calidad del catálogo. Una organización puede garantizar que todo el contenido disponible en su instancia cumple sus estándares de calidad y está correctamente catalogado.

SoundCloud

SoundCloud es la plataforma de referencia para artistas independientes. Desde su fundación en 2007, ha permitido a músicos de todo el mundo subir y distribuir su contenido de forma gratuita, lo que la ha convertido en el punto de origen de géneros como el cloud rap o el hyperpop. El enfoque centrado en el creador, con control directo sobre el contenido publicado, es lo que más se acerca al espíritu de MusicPlayer.

Sin embargo, SoundCloud es todavía una plataforma centralizada y de código cerrado. La experiencia del oyente es inferior a la de Spotify y el sistema de roles es binario: artista o oyente. No existe la figura del editor ni del administrador del sistema con permisos diferenciados.

Lo que MusicPlayer aporta frente a SoundCloud: un sistema de roles más granular que permite a una organización gestionar tanto el catálogo general como el contenido de artistas individuales con permisos diferenciados.

Deezer

Deezer es una plataforma francesa con especial presencia en Europa y América Latina. Destaca por su catálogo de más de 90 millones de canciones, las letras sincronizadas en tiempo real y el audio lossless en formato FLAC en su plan HiFi. La funcionalidad de letras sincronizadas es especialmente relevante como referencia para MusicPlayer: mientras Deezer muestra la línea que se está cantando resaltada en tiempo real, MusicPlayer muestra el texto completo sin sincronización temporal. Ampliar esa funcionalidad se identifica como trabajo futuro.

Tidal

Tidal es una plataforma fundada en 2014 con una propuesta de valor diferenciada: la calidad del audio. Ofrece streaming en formato FLAC lossless y, en su nivel más alto, audio espacial en Dolby Atmos y Sony 360 Reality Audio.

También se distingue por un modelo de compensación a artistas que afirman es más justo que el de Spotify, aunque este argumento ha sido cuestionado en varias ocasiones por análisis independientes.

Desde el punto de vista técnico, es un sistema propietario como los demás. El precio de la suscripción es más alto, lo que limita su base de usuarios.

Lo que MusicPlayer aporta frente a Tidal: accesibilidad económica y apertura. No hay coste de licencia y la calidad del audio depende únicamente de los archivos que el operador decida almacenar.

Alternativas de código abierto: Navidrome y Ampache

Además de las plataformas comerciales, existen alternativas de código abierto que vale la pena mencionar porque se acercan más al nicho que ocupa MusicPlayer.

Navidrome es un servidor de música de código abierto compatible con el protocolo Subsonic. Permite organizar y reproducir una biblioteca musical alojada en el propio servidor y se puede consumir desde una gran variedad de clientes. Es una herramienta consolidada para gestionar música personal, pero no tiene un sistema de roles orientado a organizaciones ni un modelo de gestión de catálogo colaborativo.

Ampache es otro servidor de música de código abierto con más de veinte años de historia. Ofrece gestión de biblioteca, streaming y una API compatible con Subsonic y AmpacheAPI. Tiene más funcionalidades que Navidrome, incluyendo soporte para vídeo y podcasts, pero su interfaz de usuario está desactualizada y su configuración es considerablemente más compleja.

Lo que MusicPlayer aporta frente a Navidrome y Ampache: una pila tecnológica moderna (Spring Boot 4 + Angular 21), una API REST contemporánea con documentación clara, un sistema de roles diseñado para organizaciones con múltiples colaboradores y una interfaz de usuario actualizada.

Resumen comparativo

Característica	Spotify	SoundCloud	Navidrome	MusicPlayer
Autogestión	No	No	Sí	Sí
Código abierto	No	No	Sí	Sí
Control de catálogo	No	Parcial	Total	Total
Sistema de roles	No	Artista/Oyente	Básico	4 roles
Letras integradas	Premium	No	No	Tiempo real
API REST moderna	Sí (cerrada)	Sí (cerrada)	Subsonic	Sí (abierta)

El nicho que ocupa MusicPlayer es claro: una plataforma autogestionable, de código abierto y con control granular de roles, orientada a organizaciones que quieren gestionar su propio catálogo musical sin depender de plataformas comerciales.

B1.4 Elicitación de Requisitos y Análisis de Riesgos

Requisitos funcionales

Tras analizar los objetivos del proyecto, identifiqué los siguientes requisitos funcionales:

ID	Requisito	Rol principal
----	-----------	---------------

RF-01	El sistema debe permitir el registro de nuevos usuarios con activación por correo electrónico	Público
RF-02	El sistema debe autenticar usuarios mediante usuario y contraseña, devolviendo un token JWT	Todos
RF-03	El sistema debe permitir la gestión completa (CRUD) de canciones, incluyendo título, duración, URL de archivo, portada y fecha de lanzamiento	ADMIN, EDITOR
RF-04	El sistema debe permitir la gestión completa de álbumes con tipo (ALBUM, SINGLE, EP)	ADMIN, EDITOR
RF-05	El sistema debe permitir la gestión completa de artistas, incluyendo el campo de verificación	ADMIN, EDITOR, ARTIST
RF-06	El sistema debe permitir la gestión completa de géneros musicales	ADMIN, EDITOR
RF-07	El sistema debe permitir a los oyentes crear y gestionar listas de reproducción propias, pudiendo marcarlas como públicas o privadas	USER
RF-08	El sistema debe permitir añadir canciones a playlists con una posición determinada	USER
RF-09	El sistema debe registrar el historial de reproducciones por usuario	USER
RF-10	El sistema debe permitir marcar canciones como favoritas	USER
RF-11	El sistema debe mostrar un reproductor visual persistente con controles de play/pausa, siguiente, anterior, volumen, aleatorio y repetición	Todos
RF-12	El sistema debe obtener y mostrar letras de canciones en tiempo real desde una API externa	Todos
RF-13	El sistema debe mostrar dashboards diferenciados según el rol del usuario autenticado	Todos
RF-14	El sistema debe enviar correo de activación al registrar un usuario y correo de recuperación al solicitar restablecimiento de contraseña	Público

Requisitos no funcionales

ID	Requisito	Métrica
RNF-01	Seguridad: todos los endpoints de negocio deben requerir autenticación JWT válida	Token HS512, 30 min de validez
RNF-02	Control de acceso: las operaciones de escritura deben estar restringidas al rol apropiado	Anotaciones @PreAuthorize en todos los endpoints
RNF-03	Responsividad: la interfaz debe funcionar correctamente en pantallas desde 375px	Bootstrap 5 grid + media queries SCSS

RNF-04	Trazabilidad: todas las entidades de negocio deben registrar quién las creó y cuándo, y quién las modificó por última vez	Spring Data JPA Auditing
RNF-05	Mantenibilidad: las migraciones de base de datos deben ser versionadas y reproducibles	Liquibase changelogs
RNF-06	Internacionalización: la interfaz debe soportar al menos español e inglés	ngx-translate con archivos JSON

Análisis de riesgos

Durante el análisis inicial identifiqué varios riesgos que podían comprometer el desarrollo o el resultado final. Para cada uno tomé decisiones de diseño específicas.

Riesgo 1: Expiración del token JWT durante la sesión. Si el token expira mientras el usuario navega, la siguiente petición al backend falla con un 401. Sin un mecanismo de gestión explícito, esto resulta confuso para el usuario, que simplemente ve que la aplicación "deja de funcionar". La decisión fue implementar `authExpiredInterceptor` en el frontend, que intercepta las respuestas 401 y redirige automáticamente al usuario a la página de login con un mensaje claro. Esto convierte un error técnico en un comportamiento esperado y manejado.

Riesgo 2: Disponibilidad de la API externa de letras. `lyrics.ovh` es un servicio gratuito de terceros sin garantías de disponibilidad. Si el servicio cae, la funcionalidad de letras deja de estar disponible. La decisión fue tratar las letras como funcionalidad opcional: el panel muestra un mensaje de error amigable cuando la API no responde, sin afectar al resto de la aplicación. Durante las pruebas comprobé que el servicio tiene tiempos de respuesta variables y que en algunos momentos tarda más de cinco segundos. Se documentó como área de mejora futura.

Riesgo 3: Rendimiento de consultas ManyToMany en JPA. La relación entre `Song` y `Artist` es ManyToMany. Cuando Hibernate carga colecciones tipo *bag* con `JOIN FETCH`, genera un producto cartesiano en la consulta SQL que puede devolver filas duplicadas y degradar el rendimiento en catálogos grandes. Este riesgo lo identifiqué al estudiar la documentación de JHipster, antes de que apareciese como problema real. La solución fue implementar `SongRepositoryWithBagRelationships`, que carga los artistas asociados en una consulta separada y los combina en memoria.

Riesgo 4: Conflicto de maquetación entre sidebar, navbar y player bar. Tres elementos de posición fija en tres bordes distintos de la pantalla pueden provocar que el contenido principal quede parcialmente oculto, especialmente en pantallas pequeñas. Este riesgo se abordó durante el desarrollo del frontend coordinando los valores de margen y `z-index` en el SCSS global.

Riesgo 5: Integración de roles personalizados en JHipster. JHipster genera por defecto solo dos roles: `ROLE_USER` y `ROLE_ADMIN`. Añadir `ROLE_EDITOR` y `ROLE_ARTIST` implicaba modificar la base de datos, las constantes del backend y la lógica de guardas del frontend. El riesgo de introducir inconsistencias era significativo. Mi decisión fue gestionar los nuevos roles exclusivamente a través de Liquibase (base de datos) y `AuthoritiesConstants.java` (backend), sin tocar la lógica de autenticación generada por JHipster. Esta decisión demostró ser correcta: los roles nuevos funcionaron desde el primer intento sin romper nada de lo existente.

Riesgo 6: Complejidad del servidor de correo en entorno de desarrollo. Configurar un servidor SMTP real durante el desarrollo es innecesariamente complejo. La decisión fue usar Maildev, un servidor SMTP local que captura los correos sin enviarlos realmente y los muestra en una interfaz web local en `http://localhost:1025`. Esto permite verificar el contenido y el formato de los correos sin riesgo de enviarlos accidentalmente a direcciones reales.

BLOQUE 2: EJECUCIÓN DEL PROYECTO

B2.1 Diseño del Sistema

Arquitectura general

MusicPlayer sigue una arquitectura de tres capas desacopladas. El frontend Angular corre en el navegador del usuario y se comunica con el backend Spring Boot exclusivamente a través de peticiones HTTP. El backend gestiona la lógica de negocio, la seguridad y el acceso a la base de datos MySQL. Para las letras de canciones, el frontend consulta directamente la API pública `lyrics.ovh` sin pasar por el backend, ya que esa API soporta CORS de forma nativa.

```
[Navegador — Angular 21]
    ↓ HTTP / JWT
[Backend — Spring Boot 4.0.3]
    ↓ JDBC / JPA
[Base de datos — MySQL 8]

[Navegador — Angular 21]
    ↓ HTTP (CORS)
[API externa — lyrics.ovh]
```

Esta separación tiene ventajas concretas. El frontend y el backend pueden desplegarse en servidores distintos. El backend puede atender a múltiples clientes (navegador, aplicación móvil futura) sin cambios. La API REST es el contrato entre ambas capas: cualquier cambio en la implementación interna de una capa es transparente para la otra, siempre que el contrato se mantenga.

El modelo de datos

El modelo de dominio tiene diez entidades JPA de negocio. La entidad central es `Song`, que tiene relaciones con:

- `Album` (ManyToOne): una canción pertenece a un álbum
- `Genre` (ManyToOne): una canción tiene un género
- `Artist` (ManyToMany): una canción puede tener varios artistas, y un artista puede tener varias canciones

El diagrama de entidades simplificado es el siguiente:

```
Genre — Album — Song — (rel_song_artists) — Artist
                        |
                        |— PlaylistSong — Playlist — User
                        |— Play — User
                        |— Like — User
```

`Album` tiene una relación directa con `Artist` (ManyToOne) además de la relación indirecta a través de `Song`. Esto simplifica las consultas cuando se quiere mostrar la discografía de un artista sin pasar por las canciones. El campo `albumType` puede tomar los valores `ALBUM`, `SINGLE`, `EP` o `PODCAST_SERIES` mediante una enumeración Java.

La entidad `PlaylistSong` actúa como tabla de relación entre `Playlist` y `Song`, pero no es una simple tabla de unión: añade campos propios (`position`, `addedAt`) que permiten ordenar las canciones dentro de una

playlist y registrar cuándo se añadieron. Esto requirió implementarla como una entidad JPA completa con clave primaria propia.

Descripción de las capas

La **capa de presentación** (frontend, responsabilidad de mi compañero) gestiona el estado de la sesión, construye las peticiones al backend y renderiza los datos recibidos. No contiene ninguna lógica de negocio.

La **capa de negocio y API** (backend, mi responsabilidad) centraliza toda la lógica de la aplicación. Valida los datos de entrada, aplica las reglas de negocio, controla los permisos de acceso y orquesta el acceso a la base de datos a través de repositorios JPA. Expone una API REST que es el único punto de entrada al sistema.

La **capa de persistencia** almacena todos los datos. El esquema se gestiona exclusivamente mediante migraciones Liquibase versionadas, lo que garantiza que cualquier entorno puede alcanzar el mismo estado de forma reproducible.

Estructura del proyecto backend

El backend sigue la estructura de capas estándar generada por JHipster, que a su vez sigue las convenciones del ecosistema Spring:

```
src/main/java/com/musicplayer/
├─ config/           ─ Configuración de Spring (seguridad, CORS, JPA, mail)
├─ domain/          ─ Entidades JPA
├─ repository/      ─ Repositorios Spring Data JPA
├─ service/         ─ Capa de servicio con la lógica de negocio
│   └─ dto/         ─ Data Transfer Objects
│   └─ mapper/      ─ Interfaces MapStruct
├─ web/rest/        ─ Controladores REST
└─ security/        ─ Componentes de seguridad (JWT, UserDetails)
```

Esta separación tiene un propósito concreto: cada capa tiene una responsabilidad única y bien definida. Los controladores no acceden directamente a los repositorios, sino que pasan por la capa de servicio. La capa de servicio no expone entidades JPA directamente, sino que las convierte en DTOs. Los repositorios no contienen lógica de negocio, solo queries sobre la base de datos.

B2.2 Implementación

B2.2.1 Tecnologías Empleadas

JHipster 9.0.0

La primera decisión tecnológica fue usar JHipster como generador de código base. JHipster es una plataforma de generación de código open-source que crea proyectos Spring Boot + Angular con una estructura lista para producción: seguridad JWT preconfigurada, Liquibase para migraciones, MapStruct para el mapeo entre capas, Docker para el despliegue y una batería de tests de integración.

La elección se justifica por el ahorro de tiempo que supone en la fase inicial. Sin JHipster, configurar manualmente Spring Security con JWT, integrar Liquibase, definir la estructura de capas del backend y generar la estructura de módulos del frontend habría consumido semanas. JHipster entregó esa base en minutos, permitiendo dedicar el tiempo disponible del TFG a la personalización, la lógica de negocio específica del dominio musical y la integración de funcionalidades no cubiertas por el generador.

Lo que JHipster no hace es desarrollar la lógica de negocio específica del dominio. Genera entidades genéricas a partir de un modelo JDL (JHipster Domain Language), pero las relaciones complejas, los roles personalizados, las validaciones de negocio y la personalización de la interfaz son trabajo del desarrollador. En ese sentido, JHipster es una herramienta de productividad, no un sustituto de la ingeniería.

Spring Boot 4.0.3

El backend está construido con Spring Boot 4.0.3, que simplifica enormemente la configuración de una aplicación Java al proporcionar un contenedor embebido (Tomcat), autoconfiguración de beans y un sistema de gestión de dependencias maduro (Maven). Spring Boot permite arrancar un servidor REST completamente funcional con unas pocas anotaciones.

La elección de Spring Boot frente a otras alternativas como Quarkus o Micronaut se basa en su madurez, su comunidad y su integración nativa con JHipster. El ecosistema Spring (Spring Data JPA, Spring Security, Spring Mail) cubre todos los requisitos del proyecto con soluciones bien documentadas y ampliamente probadas en producción.

Angular 21

El frontend (responsabilidad de mi compañero) está construido con Angular 21 en modo standalone, la arquitectura moderna introducida a partir de Angular 14 y consolidada en versiones posteriores. En el modo standalone, los componentes no necesitan declararse en módulos NgModule, lo que simplifica la estructura del proyecto y facilita el lazy loading.

Angular 21 introduce mejoras significativas en el sistema de señales reactivas (`signal` , `computed` , `effect`), que mi compañero usó para la gestión del estado interno del reproductor y del panel de letras. Las señales ofrecen una alternativa más sencilla y eficiente que RxJS para el estado local de los componentes.

MySQL 8

Se eligió MySQL 8 como base de datos tanto en desarrollo como en producción, por su amplia adopción en el sector y la experiencia previa del equipo con el motor. A diferencia de muchos proyectos JHipster que usan H2 en memoria para el desarrollo, se tomó la decisión de trabajar directamente con MySQL desde el primer día. Esto elimina las diferencias de comportamiento entre entornos y garantiza que cualquier problema con consultas o restricciones de clave foránea se detecta durante el desarrollo, no al desplegar en producción. La configuración del entorno de desarrollo apunta a `localhost:3306/music-player` con credenciales `root/root` , y se utiliza Docker para levantar el servidor de base de datos.

HTML5 Audio API

La reproducción de audio real se implementó en el frontend usando la API de Audio nativa del navegador (`new Audio()`), sin necesidad de ninguna librería externa. El `PlayerService` de Angular gestiona el objeto `Audio` , suscribiéndose a sus eventos (`timeupdate` , `loadedmetadata` , `ended`) para mantener sincronizadas las señales de estado: progreso, duración, tiempo actual y estado de reproducción. Cuando el usuario selecciona una canción, el servicio construye la URL de streaming apuntando al endpoint `/api/upload/stream/{filename}` del backend y el navegador solicita el archivo de audio directamente. La cabecera `Accept-Ranges: bytes` que devuelve el backend permite que el navegador soporte el salto a cualquier punto del archivo sin necesidad de descargarlo completo.

Liquibase

El esquema de la base de datos se gestiona exclusivamente con Liquibase, desactivando la generación automática de DDL de Hibernate (`spring.jpa.hibernate.ddl-auto: none`). Cada cambio en el esquema se define como un `changeset` XML versionado en `src/main/resources/config/liquibase/` . Al arrancar la aplicación, Liquibase aplica automáticamente los `changesets` pendientes en el orden correcto.

Este enfoque garantiza que cualquier entorno puede alcanzar el estado correcto de la base de datos sin intervención manual. También proporciona trazabilidad completa: la historia del esquema está en el repositorio de código junto con el código que depende de él.

MapStruct

MapStruct es un generador de código que crea implementaciones de mappers entre objetos Java en tiempo de compilación. El resultado es un código de mapeo eficiente sin reflexión en tiempo de ejecución, mucho más rápido que alternativas como ModelMapper.

La separación entre entidades JPA y DTOs es un patrón de diseño que evita exponer directamente el modelo de datos interno en la API REST. Los DTOs permiten controlar exactamente qué campos se exponen en cada endpoint y añadir validaciones específicas de la capa de presentación sin contaminar las entidades JPA.

Postman

Para verificar el funcionamiento de los endpoints REST durante el desarrollo, usé Postman. Definí una colección con todos los endpoints del backend, agrupados por recurso, con variables de entorno para la URL base y el token JWT. Esto me permitió probar cada endpoint de forma independiente sin necesidad de tener el frontend arrancado, lo que aceleró considerablemente el ciclo de desarrollo del backend.

Docker Compose

JHipster genera un `docker-compose.yml` que levanta MySQL, Maildev (servidor SMTP local) y otros servicios auxiliares. El uso de Docker Compose garantiza que el entorno de desarrollo es reproducible: cualquier persona que clone el repositorio puede levantar la base de datos con un solo comando, sin necesidad de configuración manual.

Bootstrap 5 y SCSS

La maquetación del frontend usa Bootstrap 5 como base para el grid responsive y los componentes UI estándar. Las personalizaciones visuales (tema oscuro, paleta de colores, componentes del reproductor) se realizaron mediante variables SCSS que se inyectan antes de compilar Bootstrap, lo que permite modificar el tema completo desde un único archivo de configuración.

B2.2.2 Desarrollo

El diseño JDL y la generación de entidades

El punto de partida del backend fue definir el modelo de datos en JDL (JHipster Domain Language). El JDL es un lenguaje específico de dominio que JHipster usa para describir las entidades, sus campos, sus relaciones y las opciones de generación. A partir de un archivo JDL, JHipster genera automáticamente las entidades JPA, los repositorios Spring Data, los servicios, los DTOs, los mappers MapStruct, los controladores REST y los changesets Liquibase correspondientes.

El proceso de diseño del JDL no fue inmediato. La primera versión del modelo tenía deficiencias que detecté durante la implementación: `Album` no tenía referencia directa a `Artist`, lo que complicaba las consultas de discografía; `PlaylistSong` era una simple tabla de unión sin campos propios, lo que no permitía guardar la posición de cada canción. Corregir estos errores implicó regenerar partes del backend y reescribir los changesets Liquibase afectados.

La lección que saqué de ese proceso es que el diseño del modelo de datos es la decisión más importante del proyecto y la más costosa de corregir una vez que el código está generado. Vale la pena dedicar tiempo extra a revisar el modelo antes de ejecutar el generador.

La estructura de un controlador REST típico

Para ilustrar cómo está construida la capa REST, el controlador de canciones (`SongResource`) es un ejemplo representativo. Su estructura es idéntica a la del resto de controladores del dominio.

Los métodos del controlador siguen siempre el mismo patrón:

1. Anotar con `@PreAuthorize` para indicar qué roles tienen acceso
2. Anotar con el verbo HTTP correspondiente y la ruta
3. Validar la entrada (campos obligatorios, longitud, formato)
4. Delegar al servicio para la lógica de negocio
5. Devolver la respuesta apropiada con el código HTTP correcto

Por ejemplo, el endpoint de creación de una canción:

```
@PreAuthorize("hasAnyAuthority('" + AuthoritiesConstants.ADMIN
    + "', '" + AuthoritiesConstants.EDITOR + "')")
@PostMapping("/songs")
public ResponseEntity<SongDTO> createSong(
    @Valid @RequestBody SongDTO songDTO
) throws URISyntaxException {
    if (songDTO.getId() != null) {
        throw new BadRequestAlertException(
            "A new song cannot already have an ID", ENTITY_NAME, "idexists"
        );
    }
    SongDTO result = songService.save(songDTO);
    return ResponseEntity
        .created(new URI("/api/songs/" + result.getId()))
        .headers(HeaderUtil.createEntityCreationAlert(
            applicationName, true, ENTITY_NAME, result.getId().toString()
        ))
        .body(result);
}
```

El `@Valid` indica a Spring que debe ejecutar las validaciones definidas en el DTO antes de que el método se ejecute. Si alguna validación falla, Spring devuelve automáticamente un 400 Bad Request con los mensajes de error correspondientes, sin que el método llegue siquiera a ejecutarse.

La paginación se implementa en todos los endpoints de lista mediante el objeto `Pageable` de Spring Data, que el cliente controla con los parámetros de query `page`, `size` y `sort`. El controlador devuelve los resultados en el cuerpo de la respuesta y el total de registros en la cabecera `X-Total-Count`:

```
@GetMapping("/songs")
public ResponseEntity<List<SongDTO>> getAllSongs(
    @org.springdoc.core.annotations.ParameterObject Pageable pageable
) {
    Page<SongDTO> page = songService.findAll(pageable);
    HttpHeaders headers = PaginationUtil.generatePaginationHttpHeaders(
        ServletUriComponentsBuilder.fromCurrentRequest(), page
    );
    return ResponseEntity.ok().headers(headers).body(page.getContent());
}
```

El backend expone trece controladores REST bajo la ruta base `/api`. La tabla siguiente resume los recursos disponibles y sus operaciones principales:

Recurso	Ruta base	Operaciones principales
Canciones	<code>/api/songs</code>	GET (paginado), GET/{id}, POST, PUT, DELETE
Álbumes	<code>/api/albums</code>	GET (paginado), GET/{id}, POST, PUT, DELETE
Artistas	<code>/api/artists</code>	GET (paginado), GET/{id}, POST, PUT, DELETE
Géneros	<code>/api/genres</code>	GET (paginado), GET/{id}, POST, PUT, DELETE
Playlists	<code>/api/playlists</code>	GET (paginado), GET/{id}, POST, PUT, DELETE
Canciones de playlist	<code>/api/playlist-songs</code>	GET, POST, PUT, DELETE
Reproducciones	<code>/api/plays</code>	GET, POST, DELETE
Favoritos	<code>/api/likes</code>	GET, POST, DELETE
Usuarios (admin)	<code>/api/admin/users</code>	GET, POST, PUT, DELETE
Cuenta	<code>/api/account</code>	GET, POST
Autenticación	<code>/api/authenticate</code>	POST
Registro	<code>/api/register</code>	POST
Activación	<code>/api/activate</code>	GET

Seguridad: autenticación JWT con Spring Security

La autenticación sigue el flujo estándar de JWT stateless. Cuando el cliente envía credenciales al endpoint `/api/authenticate`, el backend las valida contra la base de datos a través de `DomainUserDetailsService`. Si son correctas, genera un token JWT firmado con HS512 usando el secreto configurado en `application.yml`. El token tiene una validez de treinta minutos.

La estructura del token JWT contiene:

- **sub** (subject): el login del usuario
- **auth**: los roles asignados, en formato `ROLE_ADMIN,ROLE_USER`
- **iat** (issued at): timestamp de creación
- **exp** (expiration): timestamp de expiración

El cliente almacena el token en `localStorage` e incluye el encabezado `Authorization: Bearer <token>` en todas las peticiones posteriores. En el backend, el filtro `JWTFilter` intercepta cada petición HTTP, extrae el token del encabezado, verifica su firma y su fecha de expiración, y si es válido establece el contexto de seguridad de Spring con las autoridades del usuario.

La decisión de no implementar refresh tokens fue consciente. Los refresh tokens añaden complejidad (requieren almacenamiento del lado del servidor, invalidación explícita, gestión del ciclo de vida) que no se justifica para este proyecto. Cuando el token expira, el usuario debe volver a autenticarse. El interceptor `authExpiredInterceptor` del frontend gestiona esa situación de forma transparente.

La configuración de la cadena de filtros de Spring Security fue el componente del backend que más tiempo llevó entender. JHipster genera la clase `SecurityConfiguration` con una configuración funcional, pero entender por qué cada pieza está donde está requirió leer la documentación de Spring Security con detenimiento. El punto clave es el orden de los filtros: `JWTFilter` debe ejecutarse antes del filtro de autenticación estándar de Spring, y la configuración CSRF debe desactivarse para una API stateless porque el mecanismo CSRF está pensado para aplicaciones con sesiones de servidor, no para APIs JWT.

El control de acceso por operación se implementa mediante la anotación `@PreAuthorize` en cada método de los controladores REST. Los cuatro roles del sistema son: `ROLE_ADMIN` (gestión completa del sistema), `ROLE_EDITOR` (gestión del catálogo musical), `ROLE_ARTIST` (gestión del propio contenido) y `ROLE_USER` (consumo de contenido). Los dos últimos son roles personalizados que no existen en el esquema JHipster base y fueron añadidos mediante una migración Liquibase dedicada.

Añadir roles personalizados a JHipster

Este fue el reto técnico más interesante del backend. JHipster genera por defecto una tabla `jhi_authority` con dos entradas: `ROLE_USER` y `ROLE_ADMIN`. Añadir `ROLE_EDITOR` y `ROLE_ARTIST` requirió intervenir en varios puntos del sistema de forma coordinada.

En la base de datos, un `changeset` Liquibase inserta las dos nuevas filas en `jhi_authority`:

```
<changeSet id="20260301-add-roles" author="alex">
  <insert tableName="jhi_authority">
    <column name="name" value="ROLE_EDITOR"/>
  </insert>
  <insert tableName="jhi_authority">
    <column name="name" value="ROLE_ARTIST"/>
  </insert>
</changeSet>
```

En el backend, se añaden las constantes en `AuthoritiesConstants.java`:

```
public static final String EDITOR = "ROLE_EDITOR";
public static final String ARTIST = "ROLE_ARTIST";
```

Y se usan estas constantes en todas las anotaciones `@PreAuthorize` de los controladores que requieren los nuevos roles.

Lo importante de este proceso es lo que no se modificó: no se tocó la lógica de autenticación de JHipster, ni la estructura de la tabla de usuarios, ni la generación del token JWT. Los nuevos roles se insertan en la base de datos y se asignan a usuarios exactamente igual que los roles existentes, usando la tabla de relación `jhi_user_authority`. El sistema de JHipster no necesita saber que existen nuevos roles: solo necesita que estén en la base de datos cuando un usuario con ese rol inicia sesión.

Subida y streaming de archivos: `FileUploadResource`

Uno de los componentes más relevantes del backend, y el que hace que la reproducción de audio sea real, es `FileUploadResource`. Este controlador gestiona tres operaciones:

Subida de imágenes (`POST /api/upload/image`): recibe un archivo de imagen, valida que el `Content-Type` sea `image/*`, genera un nombre único con UUID y lo almacena en el directorio configurado con

`app.upload.dir` (por defecto `uploads/`). Devuelve la URL relativa del archivo guardado, que se usa como portada de la canción o del álbum.

Subida de audio (`POST /api/upload/audio`): igual que el anterior pero para archivos de audio (`audio/*`). Devuelve la URL relativa y el nombre de archivo. En el formulario de creación de canciones, el editor sube el MP3 directamente y el campo `fileUrl` de la canción se rellena automáticamente con la URL devuelta.

Streaming de audio (`GET /api/upload/stream/{filename}`): sirve el archivo de audio al reproductor del frontend. La respuesta incluye la cabecera `Accept-Ranges: bytes`, que permite al navegador solicitar rangos de bytes concretos del archivo. Esto significa que el usuario puede saltar a cualquier punto de la canción sin que el navegador tenga que descargar el archivo completo desde el principio.

```
@GetMapping("/stream/{filename}")
public ResponseEntity<Resource> streamAudio(@PathVariable String filename) throws
IOException {
    Path uploadPath = Path.of(uploadDir).toAbsolutePath().normalize();
    Path filePath = uploadPath.resolve(filename).normalize();

    // Seguridad: evitar path traversal
    if (!filePath.startsWith(uploadPath)) {
        return ResponseEntity.badRequest().build();
    }

    Resource resource = new UrlResource(filePath.toUri());
    if (!resource.exists() || !resource.isReadable()) {
        return ResponseEntity.notFound().build();
    }

    String contentType = Files.probeContentType(filePath);
    if (contentType == null) contentType = "audio/mpeg";

    return ResponseEntity.ok()
        .contentType(MediaType.parseMediaType(contentType))
        .header(HttpHeaders.CONTENT_DISPOSITION, "inline; filename=\"" + filename +
"\")
        .header(HttpHeaders.ACCEPT_RANGES, "bytes")
        .body(resource);
}
```

La validación `filePath.startsWith(uploadPath)` es una medida de seguridad contra ataques de path traversal: si alguien intenta construir una URL con `../` para acceder a archivos fuera del directorio de uploads, el backend devuelve un 400 Bad Request sin llegar a acceder al sistema de ficheros.

La configuración de multipart en `application-dev.yml` establece un límite de 10 MB tanto para archivos individuales como para el total de la petición:

```
spring:
  servlet:
    multipart:
```



```
max-file-size: 10MB
max-request-size: 10MB
```

Endpoints adicionales y API enriquecida

Además de los endpoints CRUD estándar generados por JHipster, se implementaron varios endpoints específicos del dominio que mejoran la experiencia de uso:

Canciones:

- `GET /api/songs/by-album/{albumId}` — lista las canciones de un álbum concreto
- `GET /api/songs/public` — lista canciones accesibles sin autenticación (para vista de catálogo público)
- `GET /api/songs/my-songs` — lista las canciones del artista autenticado
- `GET /api/songs/admin` — vista completa para administrador con todos los campos

Favoritos:

- `POST /api/likes/toggle/{songId}` — alterna el estado de favorito de una canción: si no existe el like lo crea, si existe lo elimina. Esto simplifica enormemente el código del frontend, que solo necesita llamar a este endpoint sin tener que gestionar el ID del like existente.
- `GET /api/likes/my` — lista los favoritos del usuario autenticado

Playlists:

- `GET /api/playlists/my` — lista las playlists del usuario autenticado
- `POST /api/playlists/{playlistId}/songs/{songId}` — añade una canción a una playlist directamente por sus IDs, sin necesidad de construir un objeto `PlaylistSong` completo

Álbumes:

- `GET /api/albums/my` — álbumes del artista autenticado
- `GET /api/albums/public` — álbumes públicos del catálogo
- `GET /api/albums/upcoming` — álbumes con fecha de lanzamiento futura

El buscador de canciones

El buscador es uno de los componentes del frontend que mejor ilustra la integración entre las señales de Angular y el servicio de canciones. `SearchComponent` reacciona a cambios en los parámetros de la URL (campo `q`) mediante el `ActivatedRoute`, lo que permite que la búsqueda sea directamente enlazable: una URL como `/search?q=extremoduro` ejecuta la búsqueda y muestra los resultados al cargar la página.

El componente usa el `PlayerService` para iniciar la reproducción directamente desde los resultados de búsqueda, pasando la lista completa de resultados como cola de reproducción. Esto significa que al reproducir una canción desde la búsqueda, el usuario puede navegar entre todos los resultados con los botones de siguiente y anterior del reproductor.

El PlayerService: reproducción de audio real con HTML5

El `PlayerService` es el corazón de la reproducción de audio. Mantiene una instancia de `Audio` de JavaScript y la controla mediante señales de Angular. La lógica de carga y reproducción de una canción:

```
private loadAndPlay(song: ISong): void {
  this.currentSong.set(song);
  const fileUrl = song.fileUrl ?? '';
  this.audio.src = fileUrl.startsWith('/') ?
```

```

    ? fileUrl
    : ` /api/upload/stream/${encodeURIComponent(fileUrl)} `;
this.audio.load();
this.audio.play().catch(console.error);
}

```

La lógica es sencilla: si `fileUrl` ya es una ruta absoluta (empieza por `/`), se usa directamente. Si no, se construye la URL de streaming apuntando al backend. Esto permite que las canciones que tienen una URL completa (por ejemplo, de un CDN externo) se sirvan directamente, sin pasar por el backend local.

La gestión de la cola de reproducción soporta modo aleatorio y modo repetición:

```

next(): void {
    if (!this.queue.length) return;
    if (this.isShuffle()) {
        this.queueIndex = Math.floor(Math.random() * this.queue.length);
    } else {
        this.queueIndex = (this.queueIndex + 1) % this.queue.length;
    }
    this.loadAndPlay(this.queue[this.queueIndex]);
}

```

El botón de anterior tiene un comportamiento especial: si la canción lleva más de tres segundos reproduciéndose, vuelve al principio de la canción actual en lugar de ir a la canción anterior. Este es el comportamiento estándar de los reproductores modernos (Spotify, Apple Music) y mejora considerablemente la usabilidad.

El patrón repositorio con relaciones Bag

La relación ManyToMany entre `Song` y `Artist` genera un problema conocido en Hibernate que merece una explicación detallada porque afecta a cualquier proyecto que use Spring Data JPA con relaciones muchos a muchos.

Cuando Hibernate carga una colección de tipo *bag* (una lista sin orden garantizado) usando `JOIN FETCH` en una consulta JPQL, el resultado es un producto cartesiano. Si una canción tiene tres artistas y se cargan diez canciones con `JOIN FETCH`, la consulta SQL devuelve potencialmente treinta filas. Hibernate las deduplicará en memoria, pero el volumen de datos transferido desde la base de datos es innecesariamente grande y en catálogos grandes puede convertirse en un problema de rendimiento serio.

La solución fue separar la carga en dos consultas mediante `SongRepositoryWithBagRelationships`:

```

@Override
public List<Song> fetchBagRelationships(List<Song> songs) {
    return em.createQuery(
        "select song from Song song " +
        "left join fetch song.artists " +
        "where song in :songs",
        Song.class
    )
    .setParameter("songs", songs)
    .getResultList();
}

```

La primera consulta carga las canciones con sus campos simples, sin las colecciones. La segunda carga los artistas asociados únicamente para ese conjunto de canciones ya cargado. El resultado se combina en memoria. De esta forma se evita el producto cartesiano manteniendo la capacidad de cargar las relaciones de manera eficiente.

El sistema de correo electrónico y activación de cuentas

El flujo de registro con activación por correo fue uno de los aspectos del backend que más tiempo costó configurar correctamente, especialmente en el entorno de desarrollo.

Cuando un usuario se registra, el backend crea la cuenta con el campo `activated = false` y genera una clave de activación aleatoria de 20 caracteres que almacena en `activation_key`. Inmediatamente después, llama a `MailService.sendActivationEmail`, que compone un correo usando una plantilla Thymeleaf y lo envía al servidor SMTP configurado.

El correo contiene un enlace con la clave de activación embebida:

```
http://localhost:4200/activate?key=a1b2c3d4e5f6g7h8i9j0
```

Cuando el usuario pulsa ese enlace, el frontend llama al endpoint `GET /api/activate?key=...`, que busca en la base de datos el usuario con esa clave de activación, verifica que no ha expirado (la clave tiene validez de tres días), y si todo está correcto activa la cuenta estableciendo `activated = true` y borrando la clave de activación.

Para el entorno de desarrollo, se configuró Maildev con la siguiente configuración en `application-dev.yml`:

```
spring:
  mail:
    host: localhost
    port: 25
    username:
    password:
```

En producción, esta configuración se reemplaza por las credenciales de un servidor SMTP real (Gmail, SendGrid, AWS SES).

Migraciones Liquibase: evolución controlada del esquema

El esquema de la base de datos evolucionó a través de diez changesets Liquibase a lo largo del desarrollo. El orden de los changesets importa: las tablas con claves foráneas deben crearse después de las tablas a las que referencian.

El orden de creación fue:

1. Tablas de usuario estándar de JHipster: `jhi_user`, `jhi_authority`, `jhi_user_authority`
2. `genre` (sin dependencias externas)
3. `artist` (sin dependencias externas)
4. `album` (referencia a `artist` y `genre`)
5. `song` (referencia a `album` y `genre`)
6. `rel_song__artists` (tabla de relación ManyToMany entre `song` y `artist`)
7. `playlist` (referencia a `jhi_user`)
8. `playlist_song` (referencia a `playlist` y `song`)
9. `play` (referencia a `jhi_user` y `song`)
10. `jhi_like` (referencia a `jhi_user` y `song`)
11. Inserción de `ROLE_EDITOR` y `ROLE_ARTIST` en `jhi_authority`

Cada changeset incluye también datos iniciales de prueba para el entorno de desarrollo: un administrador, un usuario oyente, géneros, artistas, álbumes y canciones. Estos datos permiten que la aplicación tenga contenido desde el primer arranque sin necesidad de introducirlo manualmente.

Un aspecto que costó aprender: Liquibase registra en la tabla `databasechangelog` los changesets aplicados, identificándolos por el `id` y el `author`. Si se modifica un changeset ya aplicado, Liquibase detecta la diferencia mediante el hash del contenido y devuelve un error al arrancar. La solución correcta es nunca modificar un changeset aplicado, sino crear uno nuevo que corrija el estado anterior. Este principio de inmutabilidad de los changesets fue algo que se aprendió a las malas la primera vez que se intentó corregir un error directamente en un changeset existente.

Monitorización y logging

Spring Boot Actuator expone varios endpoints de gestión bajo `/management/`: `health` para el estado del servicio, `info` para metadatos de la aplicación, `metrics` para métricas JVM y de negocio, y `liquibase` para el estado de las migraciones. Todos estos endpoints están protegidos y solo son accesibles para `ROLE_ADMIN`.

El logging de la capa de negocio está implementado mediante AOP (Aspect-Oriented Programming) a través de `LoggingAspect`. Este aspecto intercepta todas las llamadas a métodos en los paquetes `repository`, `service` y `web.rest`, registrando automáticamente la entrada y salida de cada método, los argumentos y el resultado, así como las excepciones que se produzcan. El resultado es trazabilidad completa de cualquier operación en el sistema sin añadir una sola línea de log en el código de negocio.

La primera vez que aparece esta técnica en el código generado por JHipster puede resultar confusa. Después de leer sobre Spring AOP, el patrón es claro: en lugar de repetir `log.debug("método llamado con parámetros X")` en cada método, se define un aspecto que intercepta cualquier método que cumpla un criterio (estar en ciertos paquetes) y ejecuta código antes y después de él. Es una forma elegante de separar las preocupaciones transversales (logging, métricas, gestión de transacciones) del código de negocio.

El frontend: componentes Angular y gestión del estado

El frontend está organizado en cuatro grandes áreas. La primera son los layouts: navbar superior, sidebar lateral y player bar inferior. Estos tres componentes son persistentes y siempre visibles una vez que el usuario ha iniciado sesión. La segunda son los dashboards de inicio, uno por cada rol de usuario. La tercera son los módulos de entidades, donde cada entidad del dominio tiene sus propios componentes de lista, detalle, creación y edición. La cuarta es el área de cuenta, donde el usuario gestiona su perfil.

El reproductor visual fue el componente más complejo de implementar. Es un componente standalone que usa señales Angular para gestionar todos los estados: si está reproduciendo o pausado, el volumen actual, si el modo aleatorio está activo, si el modo repetición está activo, la posición de progreso y si el panel de letras está visible. Las señales permiten que cualquier cambio en el estado se refleje automáticamente en la vista sin necesidad de detectar cambios manualmente.

El panel de letras es un subcomponente del reproductor que inyecta `LyricsService`. Cuando el usuario activa el panel, el servicio realiza una petición GET a `https://api.lyrics.ovh/v1/{artista}/{titulo}`. El componente gestiona tres estados: cargando (spinner), letras encontradas (texto) y letras no encontradas (mensaje informativo).

La sidebar adapta su contenido según el rol del usuario. Los enlaces visibles varían: el administrador ve accesos a la gestión de usuarios y entidades del sistema, el editor ve accesos al catálogo musical, y el oyente ve accesos a sus playlists y favoritos. Esta lógica se implementa mediante el método `hasAnyAuthority()` del servicio `AccountService`, que comprueba los roles presentes en el token JWT.

Los interceptores HTTP son otro elemento clave de la arquitectura frontend. `authInterceptor` añade el token JWT a cada petición saliente. `authExpiredInterceptor` intercepta las respuestas 401 y redirige al usuario a la página de login. Ambos interceptores se registran globalmente y son transparentes para el resto del código.

Perfiles de despliegue

La aplicación tiene dos perfiles principales de Spring Boot. El perfil `dev` usa H2 en memoria, activa la consola H2 en `/h2-console`, muestra las queries SQL de Hibernate en el log y recrea el esquema en cada arranque mediante Liquibase. Es el perfil usado durante el desarrollo diario.

El perfil `prod` usa MySQL 8, desactiva los logs SQL, activa el pool de conexiones HikariCP y la caché de segundo nivel de Hibernate, y aplica una configuración CORS más restrictiva. También activa la compresión de respuestas HTTP.

Para cambiar de perfil:

```
./mvnw spring-boot:run -Dspring-boot.run.profiles=prod
```

O en Docker:

```
SPRING_PROFILES_ACTIVE=prod java -jar app.jar
```

B2.3 Pruebas del Sistema

La estrategia de pruebas del proyecto se describe en detalle en el documento independiente **PLAN_DE_PRUEBAS**, que recoge los casos de prueba organizados por módulo, las herramientas utilizadas en cada nivel de prueba y los criterios de aceptación.

A modo de resumen, la estrategia combina cuatro tipos de pruebas. Las pruebas unitarias del frontend usan Vitest y verifican la lógica de los servicios Angular de forma aislada. Las pruebas de integración del backend usan JUnit 5 y Spring Boot Test y verifican los controladores REST, los repositorios JPA y el control de acceso por rol. Las pruebas manuales funcionales cubren los flujos de usuario completos desde el navegador. Las pruebas de regresión visual comprueban la consistencia del diseño tras cada cambio significativo de estilos.

Todos los casos de prueba documentados resultaron exitosos en la versión actual del sistema.

BLOQUE 3: PLANIFICACIÓN DEL PROYECTO

Nota: El proyecto se encuentra en desarrollo activo. Las secciones B3.3 y B3.4 reflejan el estado real del desarrollo hasta la fecha de cierre de esta memoria (mayo de 2026). La versión definitiva se actualizará cuando el proyecto esté completamente finalizado.

B3.1 Planificación Temporal Inicial

El desarrollo del proyecto se planificó en seis fases secuenciales, con algunas solapadas en el tiempo.

La primera fase, de **investigación y aprendizaje**, se estimó en tres semanas. Su objetivo era familiarizarse con JHipster, estudiar la arquitectura Spring Boot + Angular que genera, entender el sistema de autenticación JWT integrado y analizar las plataformas de streaming existentes para el estado del arte.

La segunda fase, de **diseño**, se estimó en dos semanas. Comprendía el diseño del modelo de datos (entidades, relaciones, tipos), la definición de los endpoints REST, la elección del esquema de roles y el diseño preliminar de la interfaz.

La tercera fase, de **desarrollo del backend**, se estimó en cuatro semanas. Incluía la generación de entidades con JHipster, la implementación de roles personalizados, la escritura de las migraciones Liquibase adicionales, la configuración de seguridad y la verificación de los endpoints con Postman.

La cuarta fase, de **desarrollo del frontend**, fue la más larga de la estimación inicial, con seis semanas. Comprendía el diseño e implementación del tema oscuro estilo Spotify, la barra lateral, la barra del reproductor, el panel de letras, los dashboards por rol y la integración de todos los módulos de entidades.

La quinta fase, de **pruebas**, se estimó en dos semanas. Incluía la ejecución de los casos de prueba funcionales, la corrección de los fallos encontrados y las pruebas de regresión visual.

La sexta y última fase, de **documentación**, se estimó en dos semanas.

La estimación inicial total fue de **diecinueve semanas** (aproximadamente cinco meses), con inicio previsto en febrero de 2026 y entrega en junio de 2026.

Fase	Duración estimada	Semanas
Investigación y aprendizaje	3 semanas	1–3
Diseño del sistema	2 semanas	4–5
Desarrollo backend	4 semanas	6–9
Desarrollo frontend	6 semanas	10–15
Pruebas y corrección	2 semanas	16–17
Documentación	2 semanas	18–19

B3.2 Planificación Financiera Inicial

Costes de hardware y software

El proyecto no requirió ningún hardware específico más allá de los equipos de desarrollo habituales (portátiles personales ya disponibles). Tampoco requirió licencias de software de pago: todas las tecnologías utilizadas (Spring Boot, Angular, MySQL Community Edition, JHipster, Liquibase, MapStruct, Bootstrap) son de código abierto y gratuitas. Las herramientas de desarrollo (IntelliJ IDEA Community, VS Code, MySQL Workbench Community) también son gratuitas en su edición comunitaria.

El único coste de infraestructura estimado es el del servidor de producción, que en caso de despliegue en un VPS básico (2 vCPU, 4 GB RAM, 40 GB SSD) tendría un coste aproximado de 10–15 €/mes.

Concepto	Coste estimado
Hardware de desarrollo	0 € (equipos ya disponibles)
Licencias de software	0 € (todo open-source)
Servidor de producción (estimado, 5 meses)	~60 €

Total material	~60 €
-----------------------	--------------

Costes de personal

Para estimar el coste de personal se usó el modelo COCOMO básico. El tamaño estimado del proyecto en líneas de código es de aproximadamente 8.000 líneas (código propio, excluyendo el código generado por JHipster). Para un proyecto de modo orgánico, COCOMO básico establece:

```

Esfuerzo = 2,4 × (KLOC)^1,05
Esfuerzo = 2,4 × (8)^1,05 ≈ 2,4 × 8,76 ≈ 21 personas-mes

```

Sin embargo, dado que el proyecto está desarrollado por un único alumno a tiempo parcial (dedicación media estimada de 20 horas semanales durante las diecinueve semanas), el esfuerzo real es de aproximadamente 380 horas.

Tomando como referencia el coste por hora de un desarrollador junior en prácticas (15 €/h), el coste de personal estimado es:

```

380 horas × 15 €/h = 5.700 €

```

Concepto	Horas	Coste/hora	Total
Desarrollo y pruebas	330 h	15 €/h	4.950 €
Documentación	50 h	15 €/h	750 €
Total personal	380 h		5.700 €

Coste total del prototipo

Concepto	Coste
Material (hardware + software + servidor)	60 €
Personal	5.700 €
Coste total del prototipo	5.760 €

B3.3 Planificación Temporal Final (Real)

La planificación real difirió de la estimación inicial en varios aspectos. La fase de diseño se extendió más de lo previsto porque el primer modelo de datos resultó incompleto y hubo que revisarlo. Las fases de backend y frontend se solaparon más de lo esperado porque algunos componentes del backend necesitaban ajustes a medida que el frontend los consumía y aparecían requisitos que no habían quedado claros en la fase de diseño.

Estado actual del desarrollo (mayo de 2026):

Fase	Duración estimada	Duración real	Estado
Investigación y aprendizaje	3 semanas	3 semanas	✅ Completada
Diseño del sistema	2 semanas	3 semanas	✅ Completada
Desarrollo backend	4 semanas	5 semanas	✅ Completada

Desarrollo frontend	6 semanas	En progreso	 En progreso
Pruebas y corrección	2 semanas	En progreso	 En progreso
Documentación	2 semanas	En progreso	 En progreso

Las desviaciones más significativas respecto a la planificación inicial fueron:

Diseño del sistema (+1 semana): El primer modelo JDL tenía deficiencias que se detectaron al intentar implementar las primeras consultas. Corregirlo implicó regenerar varios módulos y reescribir los changesets Liquibase afectados.

Desarrollo backend (+1 semana): La implementación del sistema de roles personalizados y la configuración del servidor de correo llevaron más tiempo del estimado. Comprender en profundidad la cadena de filtros de Spring Security requirió una dedicación adicional que no se había previsto.

Desarrollo frontend y pruebas (en progreso): Estas fases siguen en curso al cierre de esta memoria. El reproductor visual está implementado, los módulos de entidades están operativos y el tema Spotify-style está aplicado. Quedan pendientes ajustes en el diseño responsive y la resolución de algunos casos de prueba de regresión visual.

El diagrama de Gantt comparativo (planificación inicial vs. real) se incluirá en la versión definitiva de esta memoria.

B3.4 Planificación Financiera Final (Real Parcial)

Las horas dedicadas realmente al proyecto hasta la fecha, desglosadas por fase:

Fase	Horas estimadas	Horas reales (a mayo 2026)
Investigación y aprendizaje	60 h	55 h
Diseño del sistema	40 h	60 h
Desarrollo backend	80 h	95 h
Desarrollo frontend	120 h	~90 h (en progreso)
Pruebas y corrección	40 h	~20 h (en progreso)
Documentación	40 h	~25 h (en progreso)
Total	380 h	~345 h (parcial)

A 15 €/h, el coste de personal hasta la fecha es de aproximadamente 5.175 €. El coste total del prototipo incluyendo infraestructura (60 €) y coste de personal parcial es de aproximadamente **5.235 €**.

La estimación final cuando el proyecto esté completo se espera dentro del rango presupuestado (5.760 €), con una desviación por encima no mayor del 5-10% debido al mayor tiempo invertido en diseño y backend respecto a lo estimado.

B3.5 Estudio de Mercado

Clientes potenciales

MusicPlayer está orientado a organizaciones que gestionan su propio catálogo musical y necesitan una plataforma de distribución autogestionada. Los segmentos de clientes potenciales más relevantes son los siguientes:

Sellos discográficos independientes. En España existen alrededor de 300 sellos discográficos independientes registrados, de los cuales un porcentaje significativo tiene catálogos propios que distribuyen a través de plataformas de terceros pagando comisiones. Una plataforma autogestionada les permitiría distribuir su catálogo directamente a sus suscriptores sin intermediarios.

Escuelas de música y conservatorios. España cuenta con más de 1.200 escuelas de música y conservatorios, que con frecuencia producen grabaciones de sus alumnos y conciertos que podrían estar disponibles para su comunidad a través de una plataforma propia.

Colectivos culturales y asociaciones. El tejido asociativo cultural español incluye miles de colectivos que organizan eventos musicales y producen contenido que no tiene cabida en las grandes plataformas comerciales.

Bibliotecas públicas con fondos de audio. Algunas bibliotecas públicas españolas cuentan con colecciones de grabaciones históricas o de música local que podrían digitalizarse y ofrecerse a los usuarios a través de una plataforma de streaming.

Tomando como universo inicial los sellos independientes y las escuelas de música, el número máximo de clientes potenciales en España se estima en aproximadamente 1.500 organizaciones. Asumiendo una tasa de adopción del 10% en los primeros dos años, el mercado objetivo inicial sería de unas 150 organizaciones.

Plan de comercialización

Si MusicPlayer se comercializara como un producto SaaS (Software as a Service) en lugar de como software open-source autoinstalable, el modelo de precios podría estructurarse como sigue:

Plan	Precio/mes	Catálogo	Usuarios	Soporte
Básico	29 €	Hasta 1.000 canciones	Hasta 100 usuarios	Email
Profesional	79 €	Hasta 10.000 canciones	Hasta 1.000 usuarios	Prioritario
Empresa	199 €	Ilimitado	Ilimitado	Dedicado

Para la producción de 150 instancias en escala (infraestructura cloud compartida), el coste de infraestructura se reduciría significativamente respecto al prototipo individual. Con un margen del 25% sobre el coste de operación y soporte, el precio del plan básico sería sostenible para la mayoría de sellos independientes y escuelas de música.

Para 150 clientes en el plan básico, los ingresos mensuales serían de 4.350 €. Los costes de infraestructura cloud compartida estimados para esa escala son de aproximadamente 800–1.200 €/mes, lo que dejaría un margen operativo bruto del 72–81% antes de costes de soporte y desarrollo continuado.

Conclusiones

MusicPlayer fue, desde el principio, una apuesta ambiciosa. Construir una plataforma de streaming musical funcional con tecnologías modernas, en el tiempo limitado de un TFG, obligó a tomar decisiones constantemente: qué implementar bien, qué simplificar y qué dejar para el futuro.

En cuanto al backend, que es la parte directamente desarrollada en esta memoria, el resultado es satisfactorio. La API REST es completa, incluyendo los endpoints de subida y streaming de audio que hacen posible la reproducción real. La seguridad funciona correctamente, el sistema de roles personalizados se integró sin romper lo que JHipster generó y las migraciones Liquibase garantizan que cualquier entorno puede reproducir el estado correcto de la base de datos. Hay cosas que se harían diferente si se empezase de nuevo: el diseño inicial del modelo JDL merece más tiempo de revisión antes de ejecutar el generador, y el tiempo dedicado a entender Spring Security al principio —en lugar de aceptar la configuración generada como una caja negra— habría evitado algún malentendido posterior.

JHipster resultó ser una herramienta mucho más útil de lo que se esperaba. No porque genere todo el código, ya que la lógica de dominio específica hay que escribirla igualmente, sino porque establece una forma de organizar el código que es coherente, escalable y bien documentada. Eso permitió centrarse en lo que importa sin tener que reinventar la rueda en cada proyecto.

La integración con la API de letras fue sencilla técnicamente, pero la dependencia de un servicio gratuito de terceros con disponibilidad no garantizada es algo que en un proyecto real habría que resolver con una solución más robusta: un servicio con SLA, una base de datos propia de letras o un mecanismo de caché local.

En cuanto a los objetivos formativos, el proyecto cumplió su propósito. Entender en profundidad cómo funciona la autenticación y la autorización en una aplicación web real, cómo se gestionan las migraciones de base de datos y cómo se diseña una API REST mantenible son conocimientos que difícilmente se adquieren de otra forma que no sea construyendo algo real.

Trabajo Futuro

El sistema actual cubre todos los requisitos funcionales definidos para el TFG, incluyendo la reproducción de audio real con streaming local. Las líneas de mejora identificadas son las siguientes:

Almacenamiento en la nube. Actualmente los archivos de audio e imagen se almacenan en un directorio local del servidor (`uploads/`). Para un despliegue en producción real, lo adecuado sería integrar un servicio de almacenamiento externo como Amazon S3, Google Cloud Storage o Cloudinary. Esto permitiría escalar el almacenamiento de forma independiente del servidor de aplicación y servir los archivos desde una CDN geográficamente distribuida.

Streaming adaptativo (HLS). El streaming actual sirve el archivo completo bajo demanda con soporte de rangos de bytes. Para conexiones lentas o archivos muy largos, el streaming adaptativo HLS (HTTP Live Streaming) permitiría ajustar la calidad del audio dinámicamente según el ancho de banda disponible, reduciendo las interrupciones.

Letras sincronizadas. Las letras se muestran actualmente como texto estático. Integrar `lrclib.net` , que proporciona letras en formato LRC con marcas de tiempo, permitiría resaltar la línea que se está cantando en tiempo real, funcionalidad presente en Spotify Premium y en Deezer.

Timeout en la petición de letras. Durante las pruebas se observó que `lyrics.ovh` puede tardar más de cinco segundos en responder. Añadir un timeout de dos o tres segundos mejoraría la experiencia del usuario sin esperar indefinidamente.

Sistema de recomendación. Con el historial de reproducciones almacenado en la base de datos, es posible construir un sistema de recomendación básico por filtrado colaborativo como microservicio independiente.

Refresh tokens. Implementarlos permitiría renovar el token de acceso sin que el usuario tenga que volver a autenticarse, mejorando la experiencia en sesiones largas.

Despliegue con CI/CD. Configurar un pipeline (GitHub Actions + Docker + VPS) convertiría el prototipo en un servicio disponible públicamente, abriendo la puerta a pruebas de usabilidad con usuarios reales.

Bibliografía

[1] JHipster Development Platform. *JHipster Documentation v9*. Disponible en: <https://www.jhipster.tech/documentation-archive/v9.0.0>

[2] Spring Framework. *Spring Boot Reference Documentation 4.0*. Disponible en: <https://docs.spring.io/spring-boot/docs/4.0.x/reference/html>

- [3] Spring Security. *Spring Security Reference: OAuth2 Resource Server*. Disponible en: <https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/index.html>
- [4] Angular. *Angular 21 Documentation: Standalone Components*. Disponible en: <https://angular.dev/guide/components/importing>
- [5] Angular. *Angular Signals Guide*. Disponible en: <https://angular.dev/guide/signals>
- [6] Liquibase Inc. *Liquibase Documentation*. Disponible en: <https://docs.liquibase.com>
- [7] MapStruct. *MapStruct Reference Guide*. Disponible en: <https://mapstruct.org/documentation/stable/reference/html>
- [8] Hibernate ORM. *Hibernate User Guide: Fetching Strategies*. Disponible en: https://docs.jboss.org/hibernate/orm/6.4/userguide/html_single/Hibernate_User_Guide.html
- [9] Bootstrap 5. *Bootstrap Documentation v5.3*. Disponible en: <https://getbootstrap.com/docs/5.3>
- [10] Spotify. *Spotify Newsroom — Company Statistics 2024*. Disponible en: <https://newsroom.spotify.com>
- [11] lyrics.ovh. *lyrics.ovh API Documentation*. Disponible en: <https://lyrics.ovh>
- [12] Boehm, B.W. *Software Engineering Economics*. Prentice-Hall, 1981. (Modelo COCOMO básico)
- [13] OWASP Foundation. *OWASP Top Ten Project*. Disponible en: <https://owasp.org/www-project-top-ten/>
- [14] Navidrome. *Navidrome Music Server Documentation*. Disponible en: <https://www.navidrome.org/docs>
- [15] Ampache. *Ampache API Documentation*. Disponible en: <https://ampache.org/api>
-

Anexo A: Glosario

API REST (Representational State Transfer Application Programming Interface). Interfaz de comunicación entre sistemas que usa el protocolo HTTP y los verbos estándar (GET, POST, PUT, DELETE) para operar sobre recursos identificados por URLs. En MusicPlayer, el backend expone una API REST que el frontend consume.

DTO (Data Transfer Object). Objeto de transferencia de datos. Patrón de diseño que consiste en crear clases específicas para transferir datos entre capas del sistema, sin exponer directamente las entidades del modelo de dominio. En MusicPlayer, los controladores REST trabajan con DTOs y los mappers MapStruct convierten entre entidades JPA y DTOs.

Guard (Angular). Clase que implementa la interfaz `CanActivate` y decide si el router puede o no navegar a una ruta determinada. En MusicPlayer, `AuthGuard` verifica la existencia y validez del token JWT antes de permitir la navegación a rutas protegidas.

JHipster (Java Hipster). Plataforma de generación de código open-source que crea proyectos completos Spring Boot + Angular/React/Vue con configuración de producción. Genera automáticamente la estructura de capas, la configuración de seguridad JWT, Liquibase para migraciones y una batería de pruebas de integración.

JWT (JSON Web Token). Estándar abierto (RFC 7519) para la transmisión segura de información entre partes como un objeto JSON firmado criptográficamente. En MusicPlayer se usa como mecanismo de autenticación stateless: el cliente incluye el token en cada petición y el servidor lo valida sin consultar la base de datos.

Liquibase. Herramienta open-source de gestión de cambios en bases de datos relacionales. Permite definir el esquema de la base de datos como una serie de changesets versionados en XML, YAML o SQL, que se aplican de forma incremental y reproducible en cualquier entorno.

MapStruct. Framework de mapeo de objetos Java que genera código de conversión entre tipos en tiempo de compilación. En MusicPlayer convierte entre entidades JPA y DTOs sin necesidad de código manual de copia de campos.

Mermaid. Lenguaje de diagramación basado en texto que permite definir diagramas de flujo, de secuencia, de clases y otros tipos de diagramas mediante una sintaxis simple. Es el formato utilizado en el documento de Diagramas de Flujo de este proyecto.

RxJS (Reactive Extensions for JavaScript). Librería de programación reactiva basada en el patrón Observer. En Angular se usa para gestionar flujos asíncronos como peticiones HTTP, eventos del router o cambios de estado que afectan a múltiples componentes.

Señal Angular (Angular Signal). Primitiva de gestión de estado reactiva introducida en Angular 16 y consolidada en versiones posteriores. Una señal es un valor que notifica automáticamente a sus consumidores cuando cambia, permitiendo actualizaciones de vista sin necesidad de la detección de cambios tradicional de Angular.

Spring Boot. Framework de desarrollo de aplicaciones Java que simplifica la configuración de Spring eliminando la necesidad de XML de configuración y proporcionando un servidor embebido. Permite crear aplicaciones ejecutables autocontenidas con configuración mínima.

Spring Security. Módulo del ecosistema Spring que gestiona la autenticación y la autorización en aplicaciones Java. En MusicPlayer se configura como servidor de recursos OAuth2, validando el token JWT en cada petición y aplicando las reglas de acceso definidas mediante anotaciones `@PreAuthorize`.

Streaming. Técnica de transmisión de contenido multimedia que permite al usuario empezar a consumir el contenido antes de que la descarga completa haya finalizado. En MusicPlayer, el streaming de audio está implementado mediante el endpoint `/api/upload/stream/{filename}` con soporte de rangos de bytes (`Accept-Ranges: bytes`), lo que permite al navegador saltar a cualquier punto de la canción sin descargar el archivo completo. El streaming adaptativo HLS, que ajustaría la calidad del audio dinámicamente según el ancho de banda, queda como mejora futura.

Anexo B: Guía de Despliegue

Esta sección describe los pasos para desplegar MusicPlayer en un entorno de desarrollo local y en un servidor de producción básico.

Requisitos previos

- Java 21 o superior
- Node.js 20 o superior con npm
- MySQL 8 (para producción) o Docker (para desarrollo)
- Maven 3.9 o superior

Despliegue en entorno de desarrollo

1. Clonar el repositorio:

```
git clone https://github.com/usuario/musicplayer.git
cd musicplayer
```

2. Levantar la base de datos y servicios auxiliares con Docker:

```
docker compose -f src/main/docker/app.yml up -d mysql
docker compose -f src/main/docker/maildev.yml up -d
```

3. Arrancar el backend:

```
./mvnw spring-boot:run -Dspring-boot.run.profiles=dev
```

El backend estará disponible en `http://localhost:8080` .

4. Arrancar el frontend:

```
npm install
npm start
```

El frontend estará disponible en `http://localhost:4200` .

5. Acceder a la aplicación: Con el perfil `dev` , la base de datos incluye datos de prueba. Las credenciales predeterminadas son `admin/admin` para el administrador y `user/user` para el oyente.

6. Ver los correos enviados: Acceder a `http://localhost:1080` para ver la interfaz de Maildev con los correos capturados.

Despliegue en producción

1. Construir el artefacto:

```
./mvnw -Pprod clean verify
```

Esto genera un `.jar` ejecutable en `target/` .

2. Configurar las variables de entorno en el servidor:

```
export SPRING_DATASOURCE_URL=jdbc:mysql://localhost:3306/musicplayer
export SPRING_DATASOURCE_USERNAME=musicplayer_user
export SPRING_DATASOURCE_PASSWORD=contraseña_segura
export SPRING_MAIL_HOST=smtp.gmail.com
export SPRING_MAIL_PORT=587
export SPRING_MAIL_USERNAME=correo@gmail.com
export SPRING_MAIL_PASSWORD=contraseña_aplicacion
export JHIPSTER_SECURITY_AUTHENTICATION_JWT_BASE64_SECRET=clave_base64_min_512_bits
```

3. Ejecutar la aplicación:

```
java -jar target/musicplayer-0.0.1-SNAPSHOT.jar \
  --spring.profiles.active=prod
```

4. Configurar Nginx como proxy inverso:

```

server {
    listen 80;
    server_name musicplayer.midominio.com;

    root /var/www/musicplayer/frontend;
    index index.html;

    location /api {
        proxy_pass http://localhost:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location /management {
        proxy_pass http://localhost:8080;
        proxy_set_header Host $host;
    }

    location / {
        try_files $uri $uri/ /index.html;
    }
}

```

Anexo C: Referencia de Endpoints REST

Tabla completa de todos los endpoints REST del backend con su método, ruta, roles permitidos y descripción.

Método	Ruta	Roles	Descripción
POST	/api/authenticate	Público	Autenticación; devuelve token JWT
POST	/api/register	Público	Registro de nuevo usuario
GET	/api/activate	Público	Activación de cuenta por clave
GET	/api/account	Autenticado	Información de la cuenta del usuario
POST	/api/account	Autenticado	Actualizar perfil del usuario
POST	/api/account/change-password	Autenticado	Cambiar contraseña
POST	/api/account/reset-password/init	Público	Iniciar recuperación de contraseña
POST	/api/account/reset-password/finish	Público	Completar recuperación de contraseña

GET	/api/songs	Autenticado	Listar canciones (paginado)
GET	/api/songs/{id}	Autenticado	Detalle de canción
GET	/api/songs/by-album/{albumId}	Autenticado	Canciones de un álbum concreto
GET	/api/songs/public	Público	Catálogo público de canciones
GET	/api/songs/public/by-album/{albumId}	Público	Canciones públicas de un álbum
GET	/api/songs/my-songs	ARTIST	Canciones propias del artista
GET	/api/songs/admin	ADMIN	Vista completa de canciones
POST	/api/songs	ADMIN, EDITOR	Crear canción
PUT	/api/songs/{id}	ADMIN, EDITOR	Actualizar canción
DELETE	/api/songs/{id}	ADMIN, EDITOR	Eliminar canción
GET	/api/albums	Autenticado	Listar álbumes (paginado)
GET	/api/albums/{id}	Autenticado	Detalle de álbum
GET	/api/albums/my	ARTIST	Álbumes del artista autenticado
GET	/api/albums/public	Público	Álbumes del catálogo público
GET	/api/albums/upcoming	Autenticado	Álbumes con fecha de lanzamiento futura
GET	/api/albums/admin	ADMIN	Vista completa de álbumes
POST	/api/albums	ADMIN, EDITOR	Crear álbum
PUT	/api/albums/{id}	ADMIN, EDITOR	Actualizar álbum
DELETE	/api/albums/{id}	ADMIN, EDITOR	Eliminar álbum
GET	/api/artists	Autenticado	Listar artistas (paginado)
GET	/api/artists/{id}	Autenticado	Detalle de artista
POST	/api/artists	ADMIN, EDITOR, ARTIST	Crear artista

PUT	/api/artists/{id}	ADMIN, EDITOR, ARTIST	Actualizar artista
DELETE	/api/artists/{id}	ADMIN, EDITOR	Eliminar artista
GET	/api/genres	Autenticado	Listar géneros
GET	/api/genres/{id}	Autenticado	Detalle de género
POST	/api/genres	ADMIN, EDITOR	Crear género
PUT	/api/genres/{id}	ADMIN, EDITOR	Actualizar género
DELETE	/api/genres/{id}	ADMIN, EDITOR	Eliminar género
GET	/api/playlists	Autenticado	Listar todas las playlists
GET	/api/playlists/my	USER	Playlists del usuario autenticado
GET	/api/playlists/{id}	Autenticado	Detalle de playlist
POST	/api/playlists	USER	Crear playlist
PUT	/api/playlists/{id}	USER	Actualizar playlist
DELETE	/api/playlists/{id}	USER	Eliminar playlist
POST	/api/playlists/{playlistId}/songs/{songId}	USER	Añadir canción a playlist por IDs
GET	/api/playlist-songs	Autenticado	Listar canciones de playlist
POST	/api/playlist-songs	USER	Añadir canción a playlist (objeto completo)
DELETE	/api/playlist-songs/{id}	USER	Eliminar canción de playlist
GET	/api/plays	USER	Historial de reproducciones
POST	/api/plays	USER	Registrar reproducción
DELETE	/api/plays/{id}	USER	Eliminar entrada del historial
GET	/api/likes	USER	Todos los likes
GET	/api/likes/my	USER	Canciones favoritas del usuario
POST	/api/likes	USER	Marcar canción como favorita
POST	/api/likes/toggle/{songId}	USER	Alternar favorito (toggle)

DELETE	/api/likes/{id}	USER	Desmarcar canción como favorita
POST	/api/upload/image	ADMIN, EDITOR, ARTIST	Subir imagen de portada
POST	/api/upload/audio	ADMIN, EDITOR, ARTIST	Subir archivo de audio
GET	/api/upload/stream/{filename}	Público	Streaming de archivo de audio
GET	/api/admin/users	ADMIN	Listar usuarios del sistema
POST	/api/admin/users	ADMIN	Crear usuario
PUT	/api/admin/users	ADMIN	Actualizar usuario
DELETE	/api/admin/users/{login}	ADMIN	Eliminar usuario
GET	/management/health	ADMIN	Estado de salud del servicio
GET	/management/info	ADMIN	Metadatos de la aplicación
GET	/management/metrics	ADMIN	Métricas JVM y de negocio
GET	/management/liquibase	ADMIN	Estado de las migraciones de BD